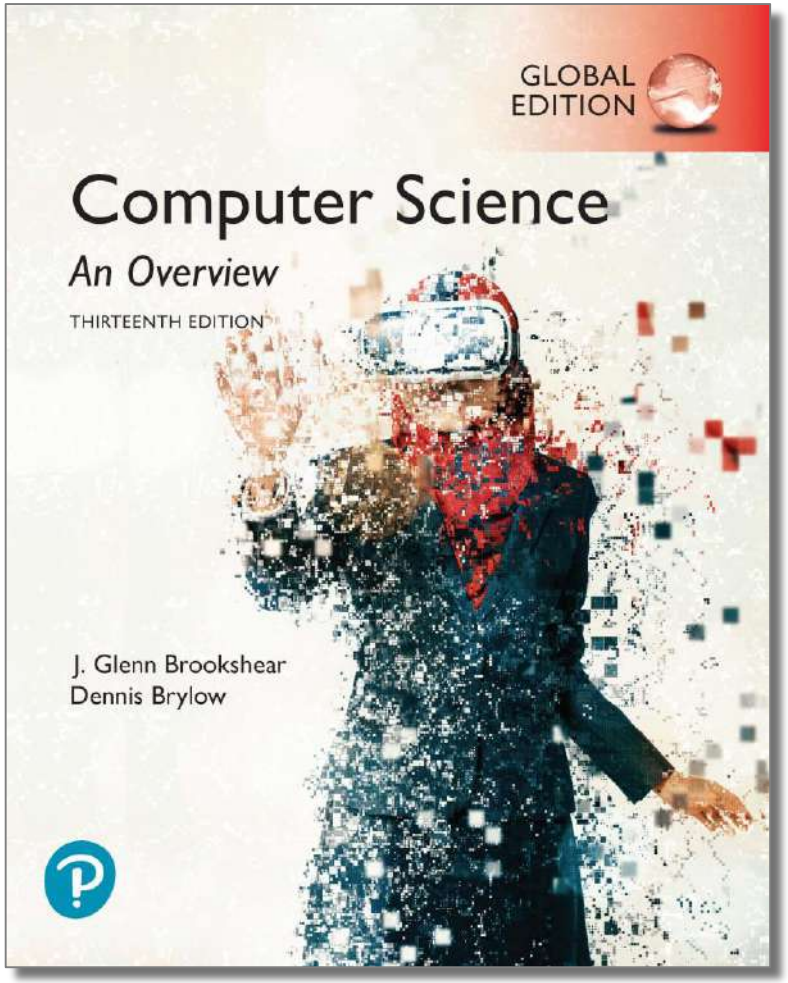
Bilgisayar Gilimine Giriş

13. Baskı, Global Edition



Bölüm 5

Algoritmalar

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

5.1 Bir algoritmanın fikri

- Bir çok araştırmacı insan beyninin her etkinliğinin bir algoritmanın sonucu olduğuna inanıyor

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Algoritmanın Resmi Tanımı

- Bir algoritma işlemi tanımlayan kesin, çalıştırılabilir ve mantıklı adımların bir araya gelmesidir
- Altgoritmanın adımları farklı şekillerde sıralanabilir
  - Doğrusal (1, 2, 3, …)
  - Paralel (çoklu işlemci)

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

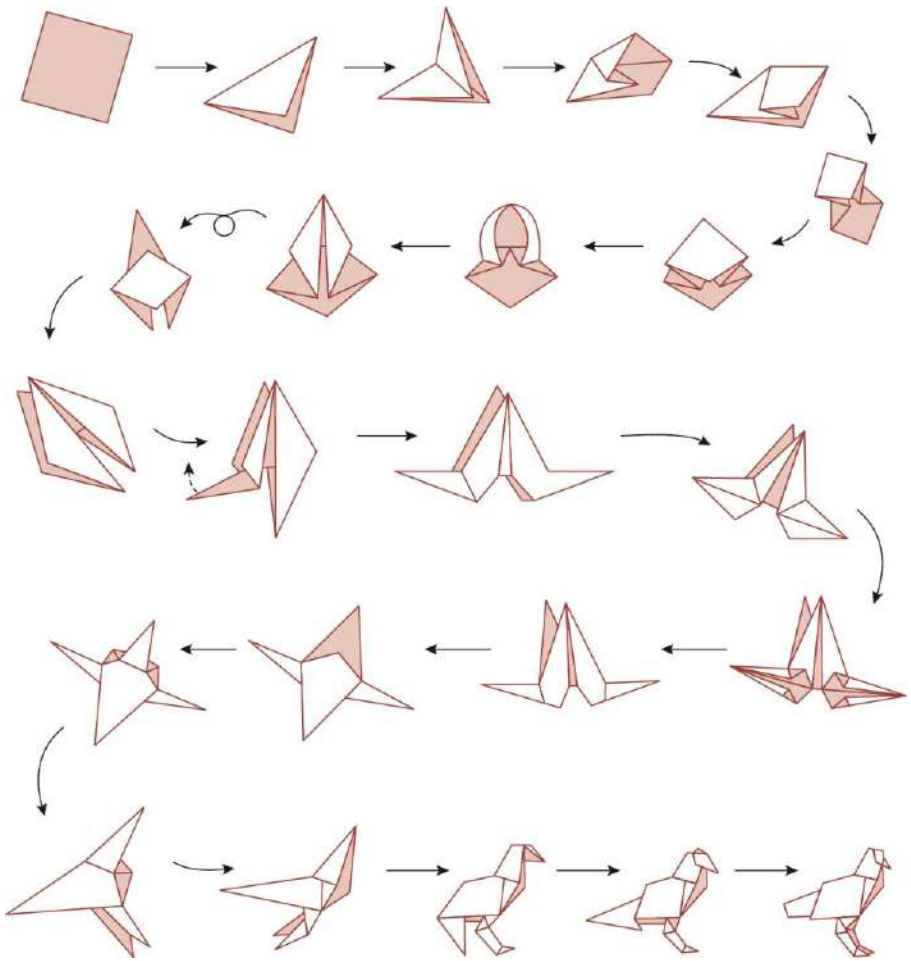
5.2 Algoritmanın Gösterimi

- İyi tanımlanmış Primitiflerle resmi bir şekilde yapılabilir
  - Bir Primitif topluluğu programlama dilini oluşturur
- Pseudocode(Sözde kod)ile resmi olmayan bir şekilde yapılabilir
  - Pseudocode konuşma diliyle kodlama dili arasında bir dildir.

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

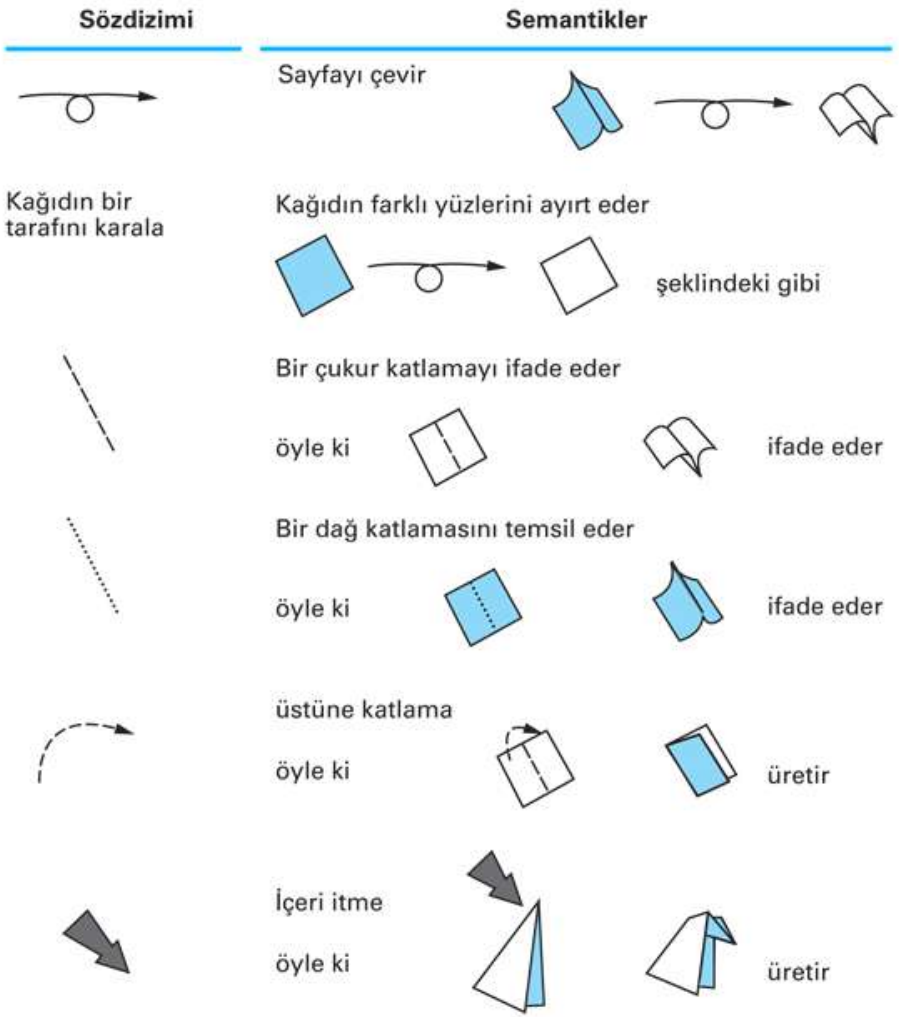
Şekil 5.2 Kare şeklinde bir kağıdı katlayarak kuş yapma



Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Şekil 5.3 Origami primitifleri



Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Bir Pseudocode Tasarlama (Sözde Kod)

- Sözde kod, bilgisayar bilimleri alanında algoritmalar ve programlar oluşturulurken ve aktarılırken kullanılan, günlük konuşma diline benzer ve belli bir programlama dilinin detaylarından uzak anlatımlardır.
- Programın yapısının ve çalışma mantığının yüksek seviyeli bir biçimde, gerektiği yerde doğrudan doğal dil cümleleriyle, ama yine de bir program yapısı ve akışı içinde anlatılmasıdır.
- Böylelikle sözde kodu okuyan ya da yazan birisi, programlama dillerinin sözdizim detaylarına dikkat etmek zorunda kalmadan, programın ve algoritmanın çalışma mantığını anlayabilir.

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Designing a Pseudocode (Sözde Kod)

Düz Yazı

1. BAŞLA
2. Birinci sayıyı gir
3. İkinci sayıyı gir
4. İki sayıyı topla
5. Sayıların toplam değerini yaz
6. BİTİR

Sözde Kod

Toplam için T, birinci sayı için X, ikinci sayı için Y seç

1. BAŞLA
2. X değerini OKU
3. Y değerini OKU
4. T = X + Y
5. T değerini YAZ
6. BİTİR

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Pseudocode Primitifleri

- Atama

```
name = expression
```

- Örnek:

```
RemainingFunds = CheckingBalance + SavingsBalance
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Pseudocode Primitifleri (devamı)

- Koşullu seçim

```
if (koşul):
    faaliyet
```

- örnek

```
if (satışlar < 100):
    fiyatı %5 düşür
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Pseudocode Primitifleri (devamı)

- Koşullu seçim

```
if (koşul):
    faaliyet
else:
    faaliyet
```

- örnek

```
if (yıl artık yıl mı):
    günlük toplam = toplam / 366
else:
    günlük toplam = toplam / 365
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Pseudocode Primitifleri (devamı)

- Tekrarlı Çalıştırma

```
while (koşul):
    gövde
```

- örnek

```
while (satılacak bilet varsa):
    bileti sat
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Pseudocode Primitifleri (devamı)

- Satırın başındaki boşluk iç içe koşulları gösterir

```
if (yağmur yağmıyor):
    if (sıcaklık == çok sıcak):
        yüzmeye git
    else:
        golf oyna
else:
    televizyon izle
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Pseudocode Primitifleri (devamı)

- Fonksiyon tanımlama

```
def isim():
```

- örnek

```
def Ahmetinisi():
```

- Fonksiyonu çağırma

```
Ahmetinisi()
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Şekil 5.4 Selamlar fonksiyonunun sözde kodu

```
def Selamlar():
    sayac = 3
    while (sayac > 0):
        print(‘Merhaba')
        sayac = sayac - 1
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

5.3 Algoritmanın keşfi

- Program geliştirmenin ilk adımıdır
- Bir yetenekten ziyade sanattır

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Polya'nın Sorun Çözme Algoritması

- 1. Problemi anla.
- 2. Problemi çözmek için bir plan yap.
- 3. Planı işleme koy.
- 4.Çözümün benzer diğer problemleri çözmek için potansiyel ve verimlilik açısından yeterli olup olmadığını kontrol et.

* matematikçi «G. Polya» 1945'te tanımlamıştır.

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Program Geliştirmede Polya'nın Algoritması

- 1. Problemi anla.
- 2. Problemi nasıl bir algoritmanın çözebileceği hakkında bir fikir edin.
- 3. Algoritmayı bul ve program haline getir.
- 4. Çözümün benzer diğer problemleri çözmek için potansiyel ve verimlilik açısından yeterli olup olmadığını kontrol et.

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Çocukların Yaşı

- A kişisi B'nin üç çocuğunun yaşlarını bulmakla görevlendirilmiştir.
  - B A'ya çocuklarının yaşının çarpımının 36 olduğunu söylüyor.
    - A başka bir bilgi daha istiyor.
  - B A'ya çocukların yaşının toplamını söylüyor.
    - A başka bir bilgi daha istiyor.
  - B A'ya en büyük çocuğun piyano çaldığını söylüyor.
    - A B'ye çocukların yaşlarını söylüyor.
- Üç çocuk kaç yaşında?

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Şekil 5.5

a. Çarpımı 36 olan üçlüler

(1,1,36)	(1,6,6)
(1,2,18)	(2,2,9)
(1,3,12)	(2,3,6)
(1,4,9)	(3,3,4)

b. (a) kısmındaki üçlülerin toplamları

1 + 1 + 36 = 38	1 + 6 + 6 = 13
1 + 2 + 18 = 21	2 + 2 + 9 = 13
1 + 3 + 12 = 16	2 + 3 + 6 = 11
1 + 4 + 9 = 14	3 + 3 + 4 = 10

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

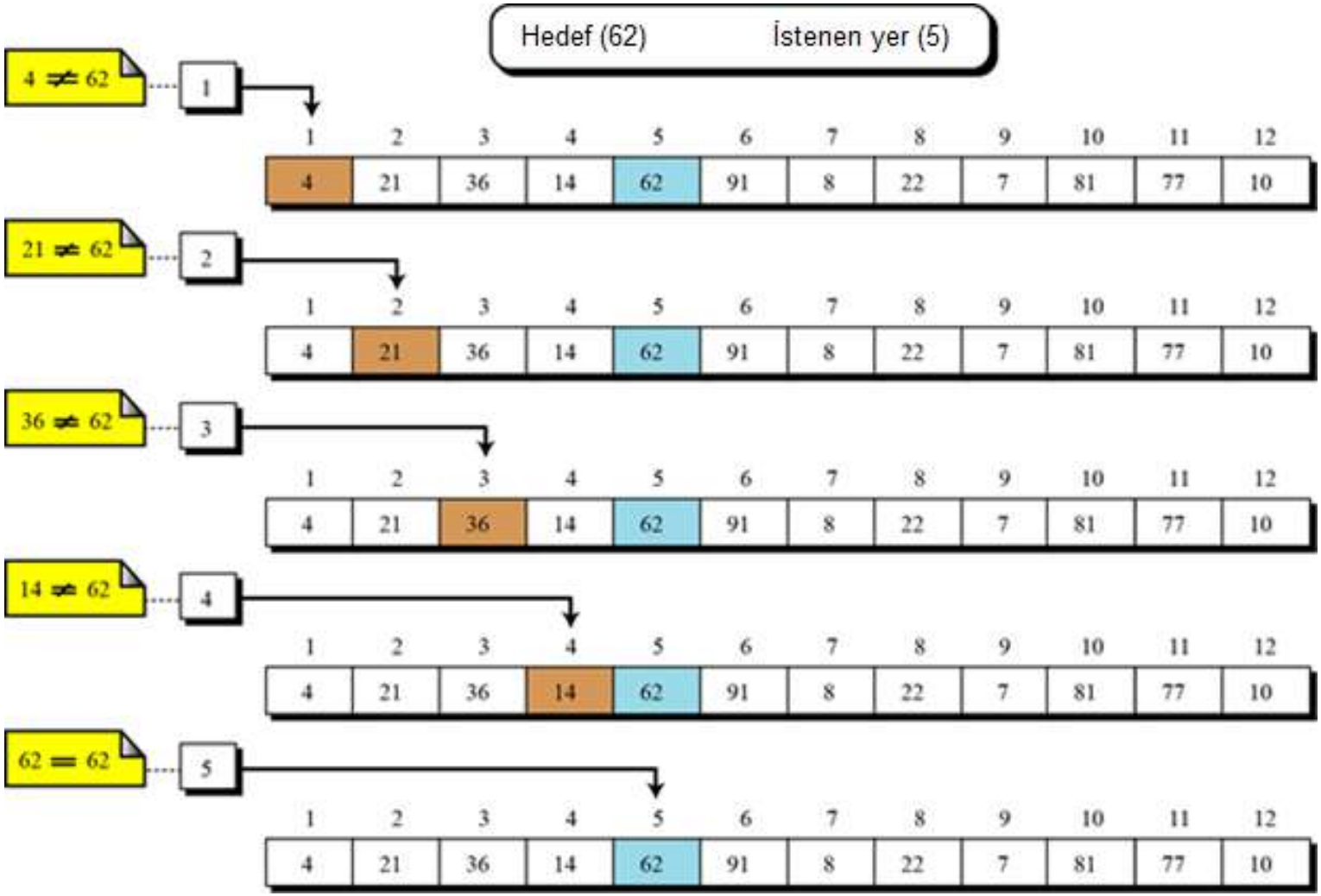
5.4 İteratif Yapılar

- Döngüsel bir yolda tekrar eden yönergeler topluluğu
- Örnek olarak:
  - Ardışık arama algoritması
  - Eklemeli sıralama algoritması

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Ardışık Arama Algoritması



Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

İteratif Yapılar

- Döngü öncesi kontrol:

```
while (koşul):
   gövde
```

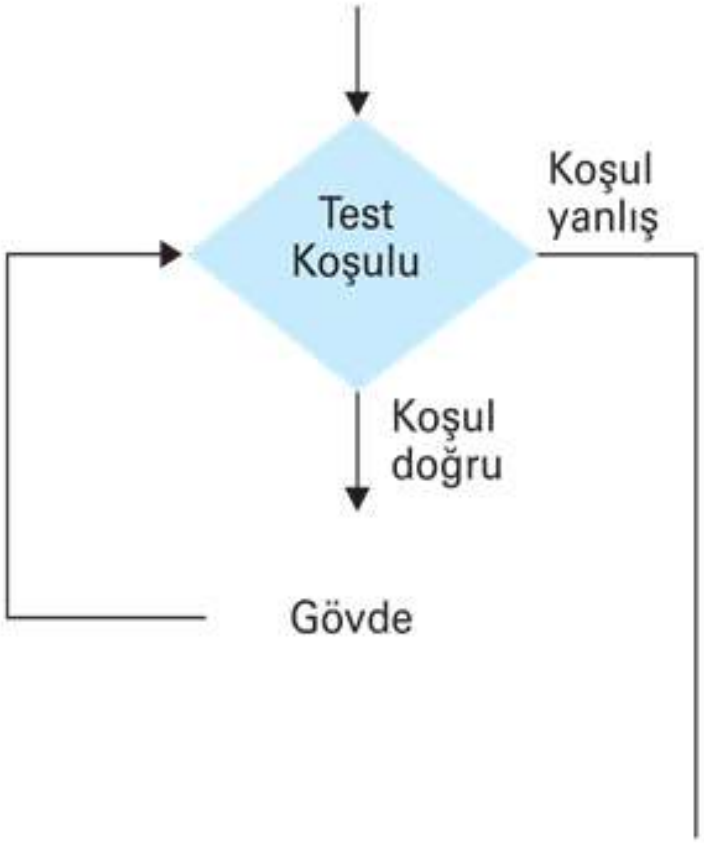
- Döngü sonrası kontrol:

```
repeat:
   gövde
   until(koşul)
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

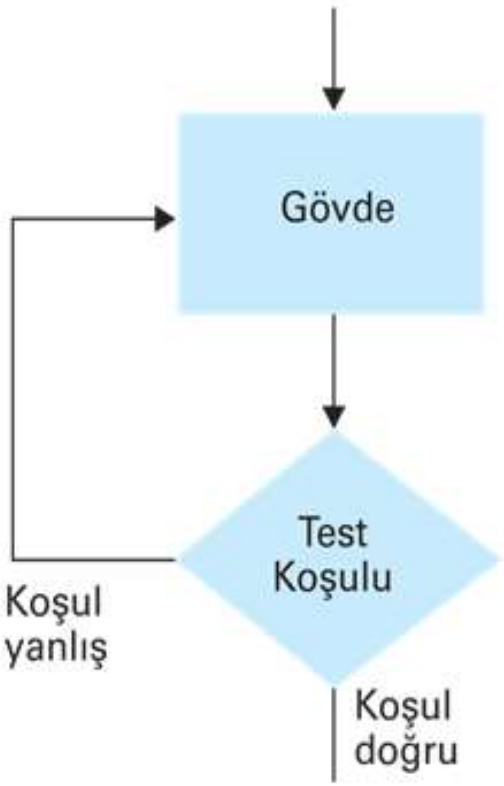
Şekil 5.8 While döngü yapısı



Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Şekil 5.9 Repeat döngü yapısı



Pearson
Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

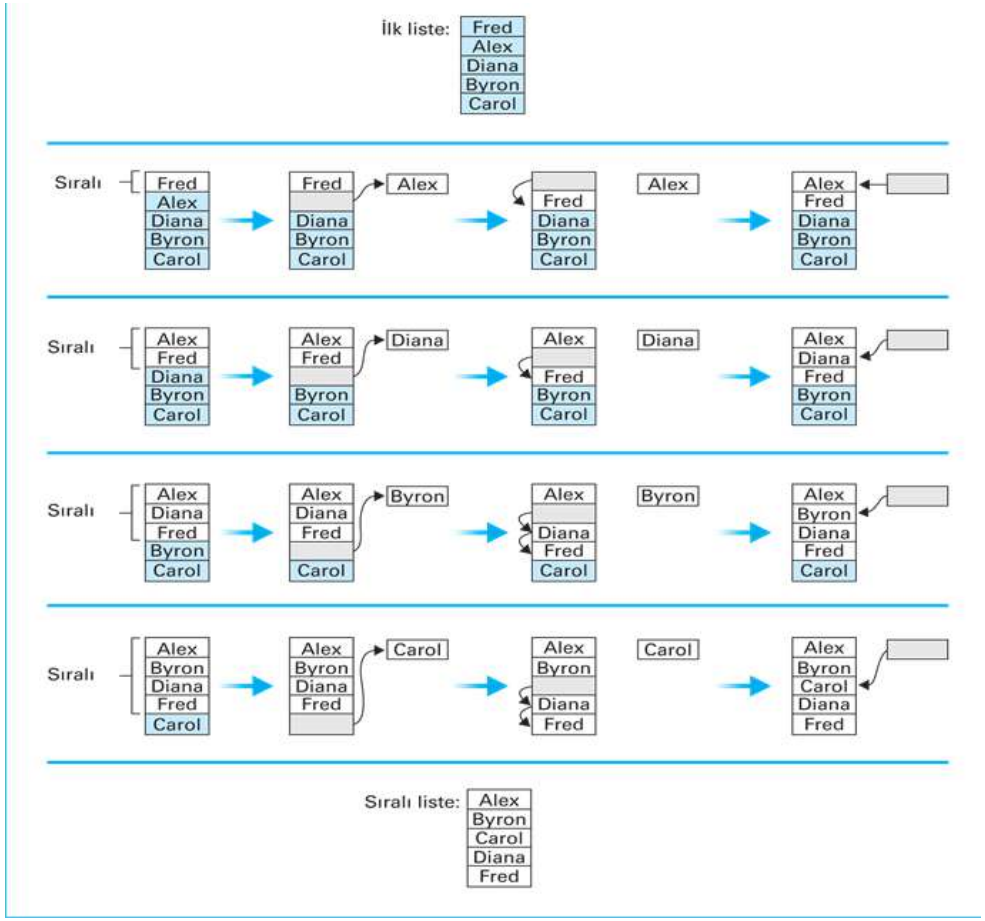
Eklemeli Sıralama (Insertion Sort)

https://visualgo.net/bn/sorting

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Şekil 5.10 Fred, Alex, Diana, Byron ve Carol isimlerini alfabetik olarak sıralayınız



Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

5.5 Özyinelemeli Fonksiyonlar

- Özyinelemeli yapılar tekrarlı faaliyetlerin gerçekleştirilmesi için döngü paradigmasına bir alternatif sunar. Bir döngü tekrarlanan bir dizi talimat içerirken, özyineleme tekrarlanan bir dizi talimatı kendisinin bir alt görevi olarak içerir.

```
#faktöriyel hesaplama

def factorial(n):
    if n <= 1 :
        return 1
    else:
        return n * factorial(n - 1)


print(factorial(3))
```

Şekil 5.12 Bir listede John elemanını aramak için stratejinin uygulanması

Liste Sıralı olmak zorundadır!

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Recursive Example:Towers of Hanoi

Find the Algorithm of Towers of Hanoi

https://www.mathsisfun.com/games/towerofhanoi.html

Solution in C Language

https://www.geeksforgeeks.org/c-program-for-tower-of-hanoi-2/

Solution in Python

https://www.geeksforgeeks.org/python-program-for-tower-of-hanoi/

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

5.6 Efficiency and Correctness

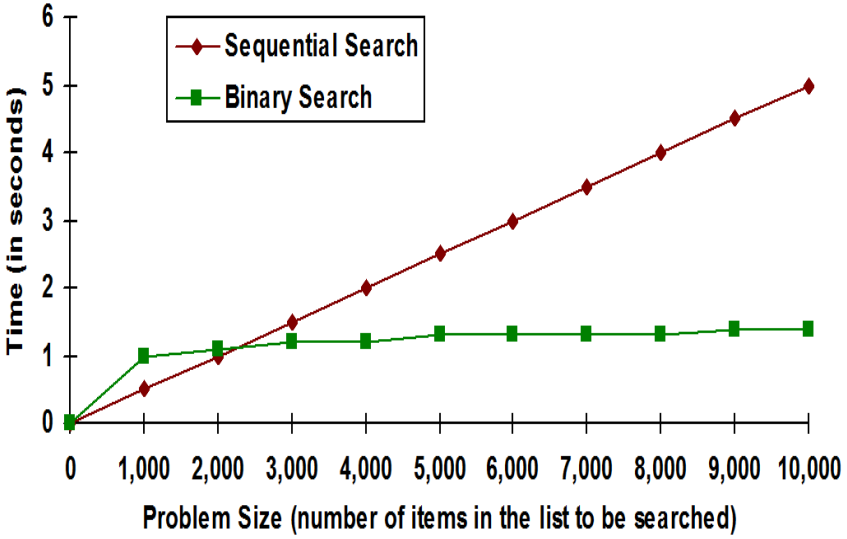
- Verimli (efficient) ve verimsiz (inefficient) algoritmalar arasındaki seçim, pratik bir çözüm ile pratik olmayan bir çözüm arasındaki farkı yaratabilir.
- Bir algoritmanın doğruluğu, algoritmanın uygulanmasını test ederek değil, resmi olarak algoritma hakkında akıl yürütme yoluyla belirlenir.

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

Algoritma Verimliliği ve Arama Yöntemleri

- **Sıralı Arama vs. İkili Arama**
- Üniversite kayıt sistemindeki 30.000 öğrenci kaydı örneği üzerinden verimlilik farkı çarpıcıdır:
  - **Sıralı Arama (Sequential Search):** Kayıtları tek tek kontrol eder. Ortalama 15.000 işlem gerekir. Her işlem 10ms sürse, sonuç için **2.5 dakika** beklenir.
  - **İkili Arama (Binary Search):** Listeyi her adımda ikiye böler. 30.000 kayıt için en kötü durumda bile sadece yaklaşık 15 işlem yeterlidir.
- Sonuç: İkili arama ile işlem süresi **0.15 saniyeye** düşer ve kullanıcıya anlık görünür.



Araya Ekleme (Insertion Sort) Analizi

Bu algoritma, bir "pivot" eleman seçip onu önceki elemanlarla karşılaştırarak doğru yere yerleştirir.

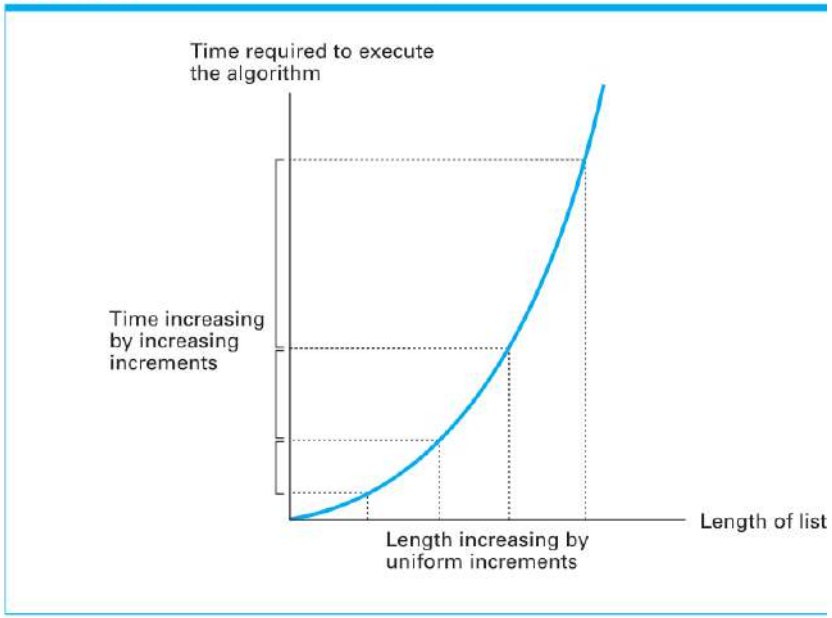
- **En İyi Durum (Best Case):** Liste zaten sıralıysa, her pivot sadece 1 karşılaştırma yapar. Toplam n - 1 işlem.
- **En Kötü Durum (Worst Case):** Liste tersten sıralıysa, her pivot kendinden önceki tüm elemanlarla karşılaştırılır.

En kötü durum formülü (karesel artış):

$$\frac{1}{2}\left(n^2 - n\right)$$

Örneğin 10 elemanlı bir liste için 45 karşılaştırma gerekirken, liste büyüdükçe maliyet katlanarak artar.

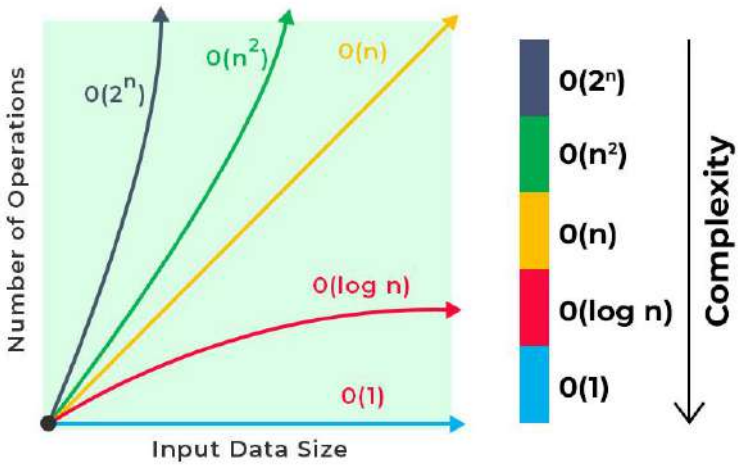
Figure 5.19 Graph of the worst-case analysis of the insertion sort algorithm



Performans Eğrileri ve Big-Theta Notasyonu

Verimlilik Sınıflandırması

Algoritmaların performansını, girdi boyutu (n) arttıkça zamanın nasıl değiştiğine göre sınıflandırırız.

- $\Theta\left(n^2\right)$: Parabolik eğri. Girdi arttıkça süre hızla artar. (Örn: Araya Ekleme Sıralaması). Büyük verilerde verimsizdir.
- $\Theta\left(\log_2 n\right)$: Logaritmik eğri. Girdi boyutu artsa bile süre çok az artar. (Örn: İkili Arama). Büyük verilerde son derece verimlidir.

Grafiklerin "şekli", algoritmanın büyük ölçekteki başarısını belirler.